

My Story Marker

Novelas personalizadas para regalar, escritas por un harness de agentes que se puede verificar

10

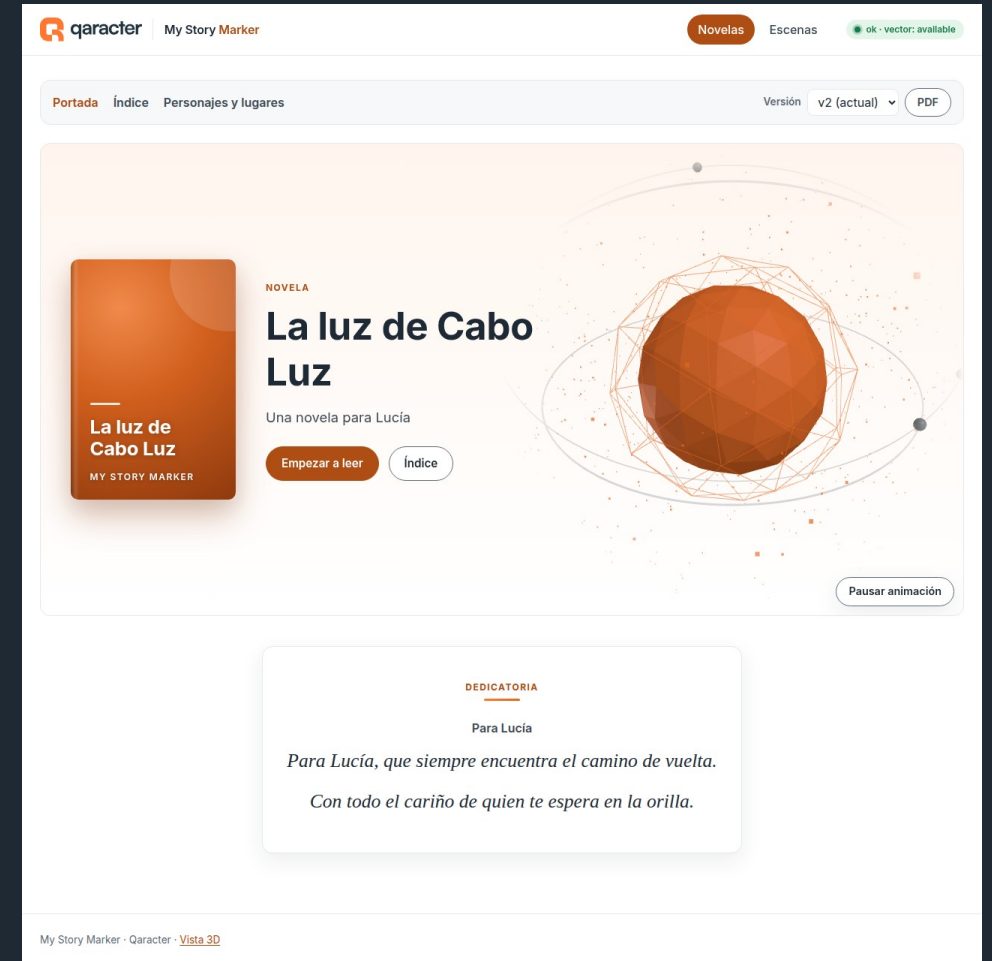
capítulos de 1.000–1.500 palabras, en español

6

roles con Claude Haiku 4.5 orquestados por código

4

tipos de validador: programático, semántico, Lean 4 y TLA+



Un regalo único que no se puede comprar hecho

El problema

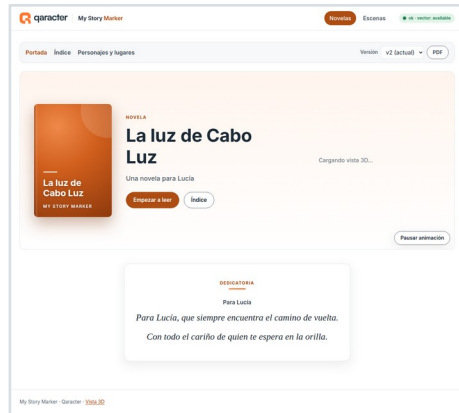
- **Escribir una novela sobre alguien** lleva meses; un regalo tiene fecha.
- **Un LLM sin harness** mezcla nombres, olvida recuerdos, contradice fechas y resucita a la mascota.
- **El cliente tiene vetos** (un nombre, un tema) que no pueden aparecer nunca.

La propuesta

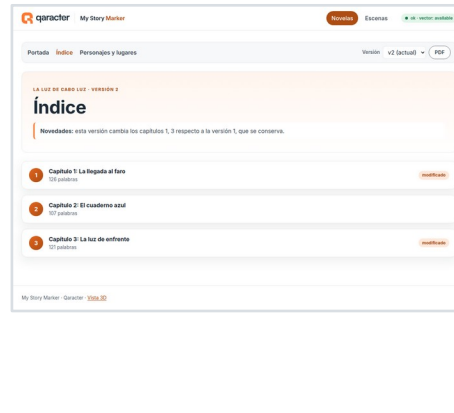
- **Entrevista guiada** → brief validado con schema; detecta datos que faltan y contradicciones.
- **Novela de 10 capítulos** en la que el destinatario se reconoce.
- **Lectura web y PDF:** portada con dedicatoria, índice, fichas de personajes.
- **Cambios puntuales:** «el perro se llama Nala» regenera solo lo necesario.

D11 • Personalización y calidad narrativa pesan igual. Que aparezcan todos los datos no basta si la historia no funciona como historia: validadores, editor y juez comprueban las dos cosas.

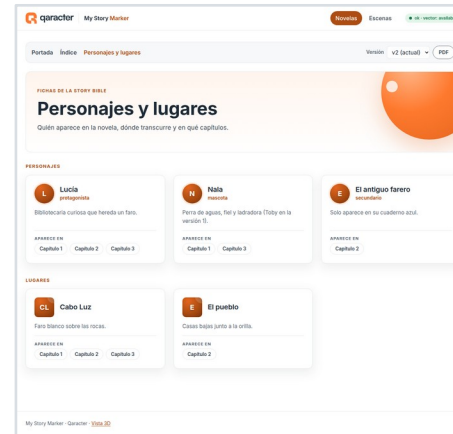
Del brief a una novela que se lee, se regala y se corrige



1 Portada con dedicatoria



2 Índice con marcas de versión



3 Fichas desde la story bible

Lector web en React y PDF exportado con la misma portada, índice navegable y fichas con enlaces «Aparece en» a cada capítulo.

El lector pide un cambio

- 1 «El perro se llama Nala» desde la página, la CLI o la API.
- 2 Un único hecho en la BD fact_usage dice qué capítulos lo usan.
- 3 Solo esos se regeneran a una versión v+1, en una transacción.
- 4 La v1 se conserva el índice marca «modificado».
- 5 PDF con «novedades» enlaces a los capítulos cambiados.

Ejecución real (ab4731f): novela de 2 capítulos → versión 2 publicada; 0 apariciones del nombre antiguo, 16 del nuevo; versión 1 intacta.

Roles con un solo trabajo, orquestados por código



Ningún modelo tiene herramientas. El orquestador (`app/novel/pipeline.py`) decide qué documentos ve cada rol, valida su salida con JSON Schema y escribe en la BD en su nombre: la tabla de permisos se aplica en código, no en un prompt. Cada fallo vuelve, con feedback y presupuesto, al rol que puede arreglarlo.

La story bible: una SQLite autoritativa con 16 tablas

1

Hechos y su uso

`fact` · `fact_usage`

Cada hecho del brief es una fila; `fact_usage` lo liga a (versión, capítulo, escena). Así `change_fact` sabe qué capítulos regenerar.

2

Cronología

`chronology_event` · `event_participant`

Fechas, lugares, participantes, edades declaradas y salidas (muerte, partida). Es la entrada del modelo Lean.

3

Versiones y checkpoints

`novel_version` · `chapter_version` · `checkpoint`

Texto + hash por capítulo y versión; texto y checkpoint en una transacción. Reanudación en el primer capítulo incompleto.

4

Auditoría y coste

`validator_result` · `policy_decision` · `llm_call`

Cada validador, decisión de política y llamada con tokens, coste, latencia y versión de prompt. Nada se borra (D6).

Los roles reciben hechos estructurados a través de tools con schema (`app/tools/`), no fragmentos recuperados por similitud. Diagrama ER en anexo-arquitectura.pdf.

Cada fallo se detecta donde es barato repararlo



Tipo	Validadores	Punto de ejecución	Si falla
Programático	brief_schema · schema_role_output · chapter_length · exact_names · brief_coverage · no_placeholders · calendar_consistency · prose_repetition (linter X02) · visual_check	hook · scene_accept · chapter_close · pre_publish	vuelve al rol productor con la evidencia
Guardrail	forbidden_words_scene · forbidden_words_chapter · free_text_injection (prescan)	entrevista · scene_accept · chapter_close	reescritura ≤ 2 → forbidden_word_limit
Semántico	judge_chapter · judge_novel · revisión humana (misma rúbrica)	chapter_close · pre_publish · una novela	editor; aprueba si cada criterio ≥ 3 y media ≥ 3,5
Formal	lean_chronology (4 invariantes) · TLA+/TLC del flujo	pre_publish · desarrollo	versión bloqueada; ≤ 2 rondas de reparación de los capítulos citados

Cada validador tiene nombre y punto de ejecución, y envía su resultado a Langfuse como score. Si se agota el presupuesto, la versión queda bloqueada: nunca se publica.

Reglas que se cumplen aunque el modelo no quiera

Vetos en 3 niveles

- **Global:** sembrado por migración (1300).
- **Novela:** los vetos del cliente en el brief.
- **Léxico:** variantes extra que aporta cada validador.
- **Un acierto** vuelve al escritor, ≤ 2 veces.

Un solo normalizador

- **Plegado común** de texto y término (NFKD, sin tildes).
- **Detecta:** t0nt0 • tontooo • t.o.n.t.o • cabrones → cabrón.
- **Sin falsos positivos:** «ridículo», «tontería».
- **Palabra completa** y frases, no subcadenas.

Texto libre = no confiable

- **Prescan determinista** de inyección antes del modelo.
- **El extractor** solo devuelve hechos, y marca la sospecha.
- **Ningún rol** recibe el texto libre crudo.
- **Cada decisión** en policy_decision y en Langfuse.

Hooks de Claude Code (mismo código que el pipeline)	Resultado
Write backend/.env • valor con forma de clave de API	bloqueado (exit 2)
Bash sqlite3 data/harness.sqlite "delete ..."	bloqueado (exit 2)
Bash sqlite3 ... "select ..." • Edit backend/app/main.py	permitido (exit 0)
Capítulo con «c4br0n» o de 2 palabras	bloqueado (exit 2)

La cronología de cada novela se demuestra, no se opina

temporalOrder

Lo contado después ocurre el mismo día o más tarde.

agesCoherent

Cada edad declarada = años desde el nacimiento.

noBiLocation

Nadie está en dos lugares el mismo día.

noAfterExit

Tras una muerte o partida, esa persona no reaparece.

- **Se genera desde la story bible** y corre en pre_publish (lake build); si falla, no se publica.
- **También antes de escribir:** el plan pasa el mismo diagnóstico y se replanea.
- **Bool + Prop + teorema:** la comprobación es una prueba; decide +kernel, sin Mathlib (300 eventos en 17 s).

Lo que Lean no vio: edades (las recalcula el exportador; las vio judge_chapter) y una partida que el planner no registró como evento.

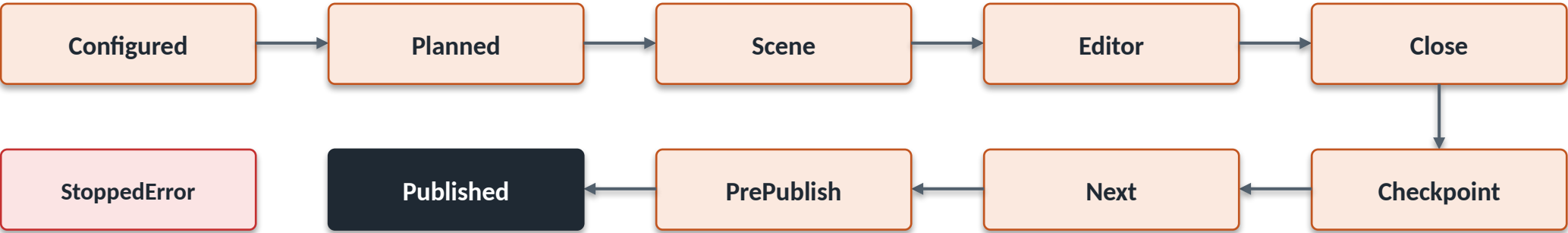
Caso real (L04)

b4-temporal, plan sin prechequeo: la mascota muere en 2005 y lleva los anillos en la boda de 2008.

Validador	¿Lo vio?
lean_chronology	Sí — noAfterExit
judge_novel	Sí — continuidad 2/5
judge_chapter	No — aprobó el capítulo de la muerte y la boda
6 validadores programáticos	No

Lectura honesta: no lo vio solo Lean; judge_novel coincidió. Lean es determinista, señala el evento exacto y actúa antes de escribir (~0,12 USD frente a ~1,3 USD). Solo prueba lo que el planner pone en la cronología.

El flujo del harness, comprobado con TLC



Bucles acotados (≤ 2 reintentos, ≤ 2 rondas de reparación) $\rightarrow$ StoppedError al agotarse. Crash $\rightarrow$ Resume desde la BD. ChangeFact $\rightarrow$ versión v+1.

Propiedad	Tipo	Garantiza
NoUnvalidatedPublish	invariante	no se publica nada que no pasó todos los validadores
ResumeNoDupNoLoss	invariante	reanudar no duplica ni pierde capítulos
PreviousVersionKept	invariante	la versión publicada anterior no cambia
RetriesBounded	invariante	reintentos acotados, también tras un crash
Termination • EveryGenerationEnds	liveness	toda generación acaba publicada o en error

Sin errores

5.492.531

estados distintos

10.031.846 generados • profundidad 127 • 08min 17s

N = 5 capítulos • 3 escenas • reintentos 2/2 • reparación 2

TLC se re-ejecutó al subir MAX_REPAIR_ROUNDS de 1 a 2 en el tuning 1. Correspondencia acción $\rightarrow$ código en `formal/tla/README.md`.

Cuatro trazas de TLC que cambiaron el código

El modelo se escribió antes que el pipeline, leyendo el esquema y el plan tal cual. TLC encontró dónde esa lectura se rompía.

	Invariante violado	Traza	Causa	Regla para el código
CE1	ResumeNoDupNoLoss	18 estados	Un crash entre la fila del capítulo y su checkpoint duplica el capítulo	Texto del capítulo y checkpoint en una sola transacción
CE2	RetriesBounded	21 estados	El contador de reintentos de capítulo vivía en memoria; un crash lo reiniciaba	Reintentos de capítulo contados desde validator_result persistidos
CE3	ResumeNoDupNoLoss	46 estados	La ronda de reparación insertaba una segunda fila del mismo capítulo	Upsert por (versión, capítulo); una versión publicada no se escribe
CE4	RetriesBounded	50 estados	El contador de rondas de reparación vivía en memoria; un crash lo reiniciaba	Ronda de reparación guardada en novel_version junto a «blocked»

Efecto: la spec de la story bible volvió a borrador y se re aprobó antes de que existiera el pipeline, que nació con las cuatro reglas. Dos mutaciones (publicar sin pre_publish, regenerar en sitio) confirman que las propiedades no son vacías.

Qué hizo cada llamada, cuánto costó y con qué prompt



Prompts versionados en las evals

Rol	Prompt	Versión	Llamadas
editor	editor	4	29
interviewer	fact_extractor	sha-609af91...	2
judge	judge_chapter	2	24
judge	judge_novel	2	7
planner	planner	4	8
writer	writer	4	45

Última versión usada por prompt. Cada llamada guarda prompt_version también en llm_call (SQLite): el antes/después no depende de Langfuse.

Cinco briefs, antes y después de la iteración de tuning

NOVELAS PUBLICADAS

0/3 → 2/3

entre los briefs generables (antes → después); b5 se rechaza en el brief, como se espera.

Brief	Antes	Después	USD
b2-infantil	bloqueada	publicada	0,97
b3-injection	bloqueada	publicada	1,02
b4-temporal	parada (plan)	bloqueada	1,93
b5-contradiction	rechazado	rechazado	—

Qué cambió: causa y arreglo

- **brief_coverage** exigía la frase literal del recuerdo → palabras de contenido normalizadas (spec 008).
- **noAfterExit** daba por «salido» a quien enterraba a la mascota → eje de la historia, solo el primer participante; Lean y Python iguales.
- **Planner** capítulos solapados y saltos vagos → time_marker por capítulo y chequeo de solape.
- **Juez** bloqueaba por defectos «posibles» → solo un defecto «alta» concreto; umbrales D11 intactos.
- **Reparación** rehacía capítulos que nadie pedía → solo capitulos_a_reparar; MAX_REPAIR_ROUNDS 1 → 2, TLC re-ejecutado.

Prompts en Langfuse: planner 2 → 4 · writer 3 → 4 · editor 3 → 4 · juez 1 → 2. b4 pasa de un falso positivo a un bloqueo correcto: su trampa de edad no tiene lectura coherente.

Red-team, informe de seguridad y los extras que suman nota

15

hallazgos en el informe X04

0

críticos o altos

5/5

medios corregidos, con test

6

corregidos en total

Red-team: 8/8 casos detenidos

X04 • Informe de seguridad

Secretos en 201 commits, pip-audit y npm audit, inyección (prescan 10/17 → 17/17), exfiltración entre novelas, hooks. Skill security-review-harness para repetirlo.

X03 • Login con SQLite

bcrypt + JWT; cada novela tiene dueño y otro usuario recibe 404. Cierra SEC-01 y SEC-08 (spec 018).

X01 • Servidor MCP

FastMCP de solo lectura: list_novels, get_chapter, list_versions, query_story_bible, download_novel. Schemas, trazas en Langfuse e identidad del usuario.

X02 • Linter de prosa

prose_repetition: repeticiones, 4-gramas y clichés de IA en español; blando para no crear bucles de reescritura.

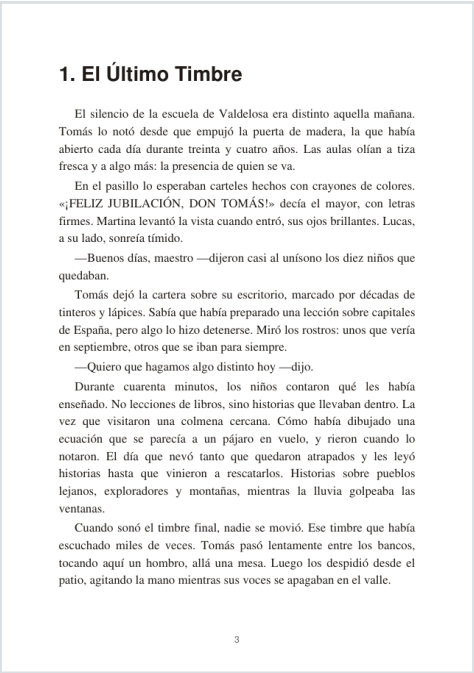
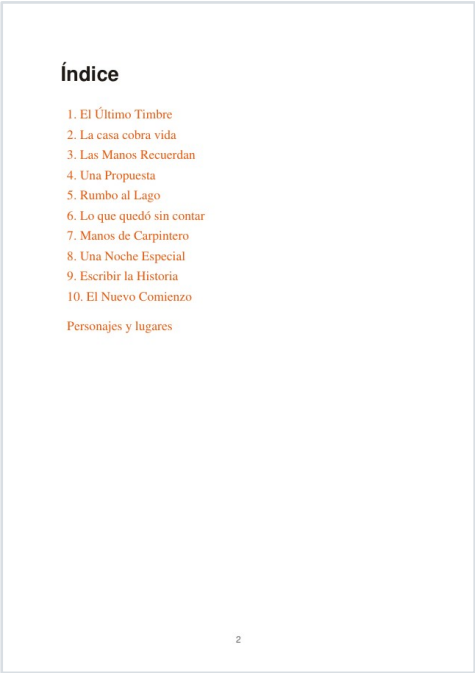
Tres paradas, una publicada: nada sin validar sale



Tuning 2 • calendario determinista. El writer inventaba el día de la semana y cada reparación inventaba otro. Ahora el plan corrige los días en Python (calendar_facts.py), writer y editor reciben plan/calendar.txt con fechas y cifras canónicas, y calendar_consistency para en chapter_close «...pero el 23 de junio de 2026 es martes».

Después • eventos con lugar. La paralela se paró por un evento del plan sin lugar que Lean no podía exportar: ahora el chequeo del plan lo rechaza y un error de exportación no gasta rondas.

Novela de 10 capítulos publicada y exportada a PDF



10
capítulos

10.645
palabras

3,10 USD
coste con Haiku 4.5

69 min
de generación

0
rondas de reparación

ejemplos/novela-ejemplo.pdf · portada, índice y capítulo 1

Brief de ejemplo del README (evals/briefs/ejemplo.json); todos los validadores en verde, incluido Lean y judge_novel.

Lo que cuesta una novela con Claude Haiku 4.5

0,35 USD

coste medio por capítulo (evals)

8 min

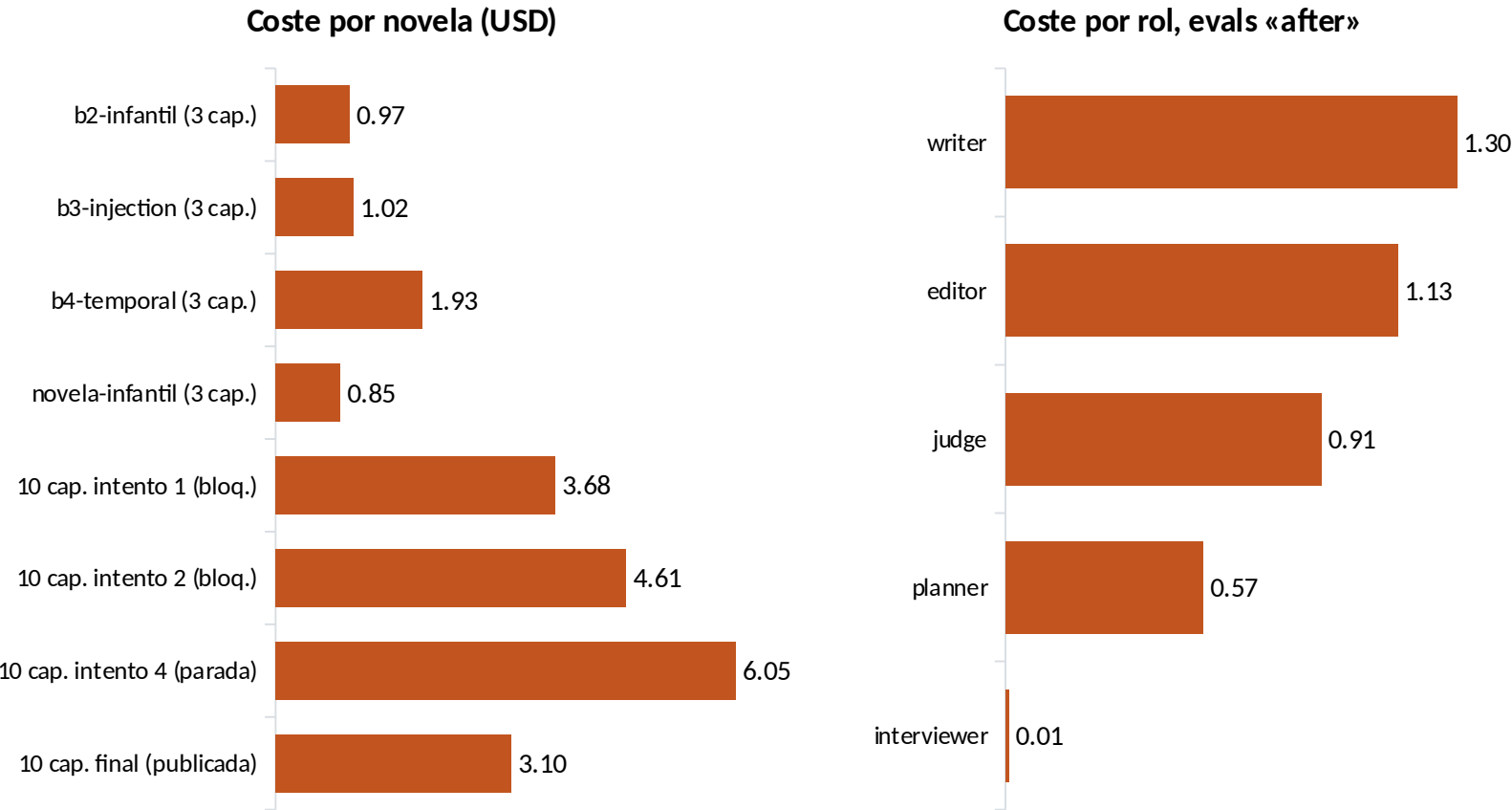
tiempo de modelo por capítulo

3,10 USD

novela de 10 capítulos

69 min

tiempo real, 10 capítulos



Datos: tabla llm_call (evals/results/**.json) y logs de la CLI (presentacion/build/data/runs.json). Las reparaciones cuentan en el coste.

Un orquestador y subagentes en paralelo, con reglas escritas

1 CLAUDE.md + AGENTS.md

Mapa de docs y Procesos 0-3: docs → spec → plan → código. Proceso 0: preguntas con recomendación antes de editar.

2 Subagentes por bloque

Un git worktree por agente, bloques en oleadas; contratos K1-K5 publicados antes que su implementación.

3 Skills

gift-novel-run y security-review-harness (creadas), react, sqlite, verification, fastapi.

4 Comandos /

/generate-novel • /inspect-novel • /change-fact • /exam-gap.

5 Hooks

PreToolUse policy_guard y PostToolUse validate_chapter: el mismo código que el pipeline, 14 casos probados.

6 Browser MCP

Playwright MCP inspeccionó el lector: cazó un 500 intermitente (SQLite entre hilos) y fichas sin versionar; ambos corregidos.

Un límite que funcionó: un subagente intentó ampliar las exenciones de una regla de seguridad y el clasificador del modo auto lo bloqueó; la decisión quedó para el humano.

Qué elegimos, frente a qué, y lo que costó

Decisión	Frente a	Por qué	Coste
Multi-agente con orquestador determinista	un agente con herramientas	permisos en código, reintento por paso, trazas por rol	más llamadas y más orquestación
SQLite autoritativa con hechos estructurados	RAG vectorial sobre la prosa	hechos exactos, fact_usage, entrada de Lean	sin búsqueda semántica en el pipeline
Calendario y cifras en Python	confiar en el LLM y el juez	determinista y barato; converge	solo días junto a una fecha
Lector web + PDF exportado	solo PDF interactivo	pedir cambios, ver versiones, validar con MCP	dos superficies que mantener
TLA+ antes que el pipeline	modelar después	4 errores de diseño cazados cuando eran baratos	el modelo abstrae la prosa
Lean core, decide +kernel	Mathlib, native_decide	sin descargas; confía en el kernel	noBilocation cuadrático
claude -p sin tools	SDK con API key	ninguna clave en el repo; rol aislado	arranque por llamada
Haiku 4.5 para todos los roles	Sonnet/Opus en writer y juez	coste y latencia de 10 capítulos	calidad vigilada por el juez
Juez bloqueante (≥ 3 , media $\geq 3,5$)	juez consultivo	D11: la calidad también bloquea	reescrituras ≤ 2 , reparaciones ≤ 2

Lo que todavía no está demostrado

Limitaciones

- **Fichas de personajes sin versionar en la BD:** solo el nombre se reconstruye por versión.
- **Días sin fecha al lado** («aquel lunes») y duraciones («treinta años») no se validan; van por prompt.
- **Lean solo prueba lo que hay en la cronología:** una partida que el planner no registra no se detecta.
- **b4 bloquea bien pero caro:** agota reparaciones y el timeout de 45 min.
- **Regla de fronteras heredada:** 21 accesos a ficheros fuera de los stores, pendientes de decisión.

Con más tiempo

- **Versionar el reparto** (descripción y rol por versión).
- **Validar días y duraciones** en la prosa, no solo junto a fechas.
- **Evento «departure» obligatorio** cuando el brief dice que alguien se fue.
- **Revisión humana** y calibración del juez con compare.py.
- **Paralelizar capítulos** independientes para bajar la latencia.
- **Tools de escritura en el MCP** con permisos y confirmación.

LA DECISIÓN DE DISEÑO MÁS IMPORTANTE

Separar la verdad (story bible SQLite + validadores deterministas + Lean/TLA+) de la prosa generada, con un bucle acotado planner → writer → editor → juez que solo publica versiones que pasan todos los validadores.

ANEXOS

[anexo-arquitectura.pdf](#) · [anexo-tla-spec.pdf](#) · [anexo-lean-invariantes.pdf](#) · [anexo-evals-tabla.pdf](#) · [anexo-revision-humana.pdf](#)